UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, *Controller*, *Model*, dan *Middleware* pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	R. A. Rafa Lutfyah
NPM	2428240125
Kelas	SI5C
Topik Tugas 1	14 - Maskapai: Jadwal Penerbangan
Entitas utama	Flight (jadwal penerbangan)
Tautan repositori	https://github.com/raden2428240125/si5c_kuis1_rafa
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	6 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]` dan `[Entitas]`, dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, `[sumber-daya]` menjadi `buku`.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan `POST`. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama `PAW2_Kuis2_SI5C_[NIM]_[Nama].pdf`, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian *endpoint*	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau `curl` di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan `Win` + `Shift` + `S`; macOS tekan `Cmd` + `Shift` + `4`; Linux gunakan `PrtSc` atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda `[...]` pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	14 - Maskapai: Jadwal Penerbangan
Entitas utama (tunggal)	Flight (jadwal penerbangan)
Alamat dasar	/flights
Field data	id, kodePenerbangan, maskapai, asal, tujuan, jadwalBerangkat
Field wajib saat POST	kodePenerbangan, maskapai, asal, tujuan, jadwalBerangkat
Query string untuk filter	?asal=CGK (filter kode bandara asal pada GET /flights)
Data awal	3 data: XY-512 (CGK-DPS), QG-880 (PLM-CGK), XY-330 (CGK-SUB)
Modul lain (jika ada)	Tidak ada (satu resource: /flights)

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey`.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/flights	Tidak	200	Ambil semua data (filter ?asal=)
2	GET	/flights/:id	Tidak	200	Ambil satu data
3	POST	/flights	Ya	201	Tambah data
4	PUT	/flights/:id	Ya	200	Ubah data
5	DELETE	/flights/:id	Ya	200	Hapus data (membalas pesan sukses dan data: null)

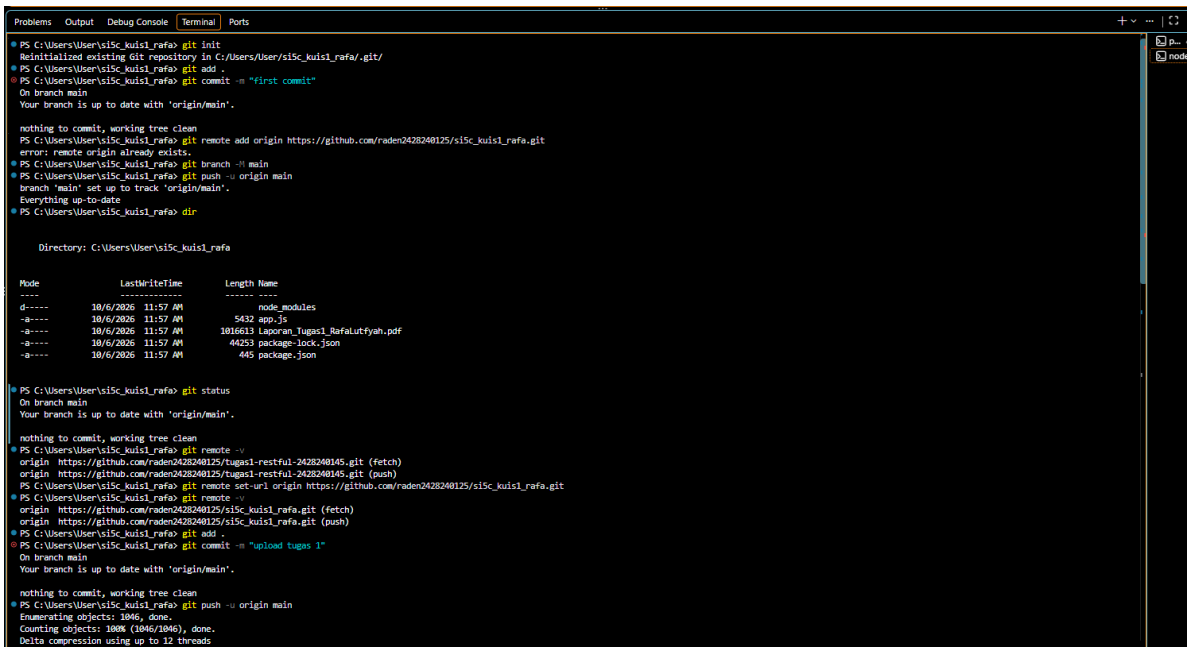
Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.



Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

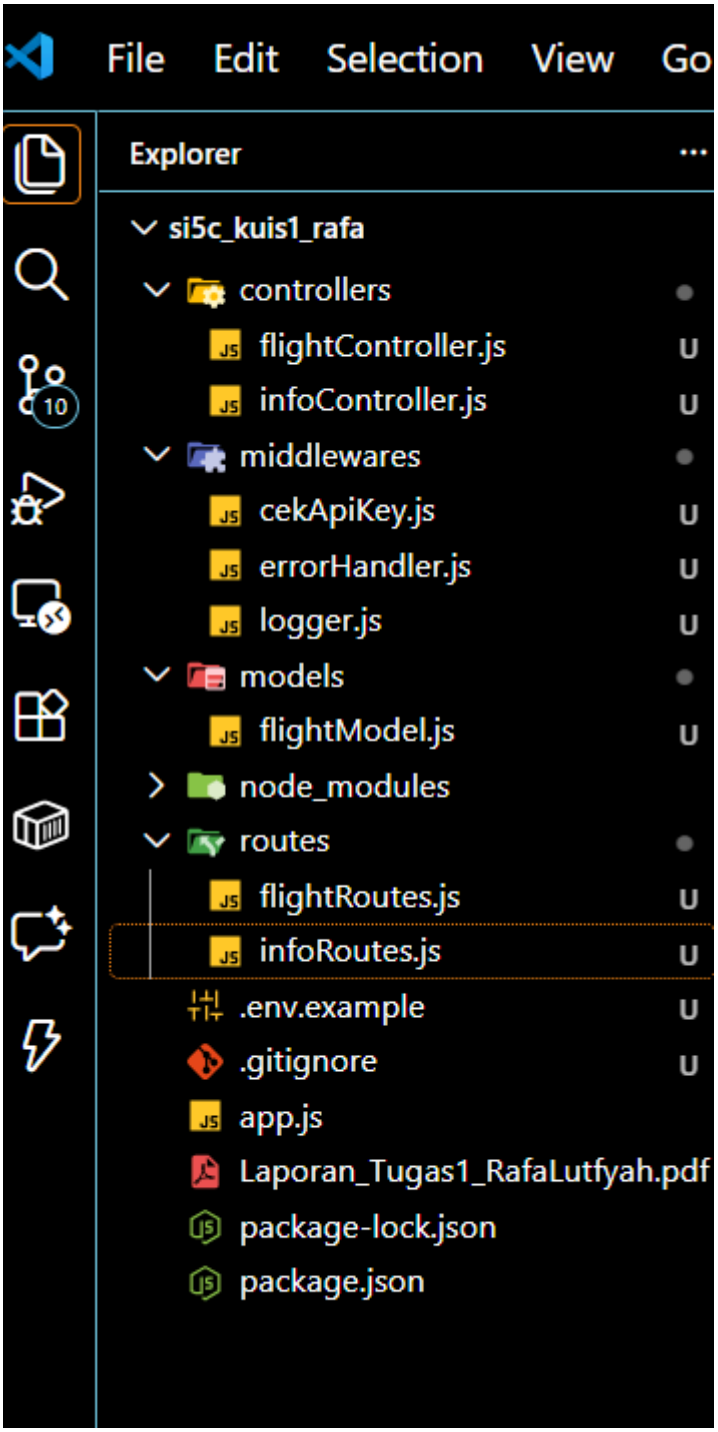
express adalah framework web untuk Node.js yang saya pakai untuk membuat server, route, dan middleware pada API jadwal penerbangan (/flights). cors saya pasang lewat app.use(cors()) supaya API bisa dipanggil dari aplikasi lain di alamat atau port yang berbeda. dotenv membaca berkas .env ke process.env, sehingga PORT dan API_KEY tidak ditulis langsung di kode. Node.js harus berversi 18 ke atas karena Express 5 mensyaratkan versi tersebut; versi yang saya pakai terlihat pada hasil node -v.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder `routes`, `controllers`, `models`, dan `middlewares` beserta isinya. Gunakan panel *Explorer* editor atau perintah `tree`.



Gambar 2. Struktur folder project

Penjelasan:
routes/ berisi pemetaan alamat /flights ke fungsi Controller; controllers/ berisi fungsi yang membaca req, memvalidasi data, dan mengirim res; models/ menyimpan array flights beserta fungsi pengolahnya tanpa memakai req dan res; middlewares/ berisi logger, cekApiKey, dan errorHandler yang bekerja sebelum atau sesudah Controller.

2.2 Ringkasan tugas setiap berkas

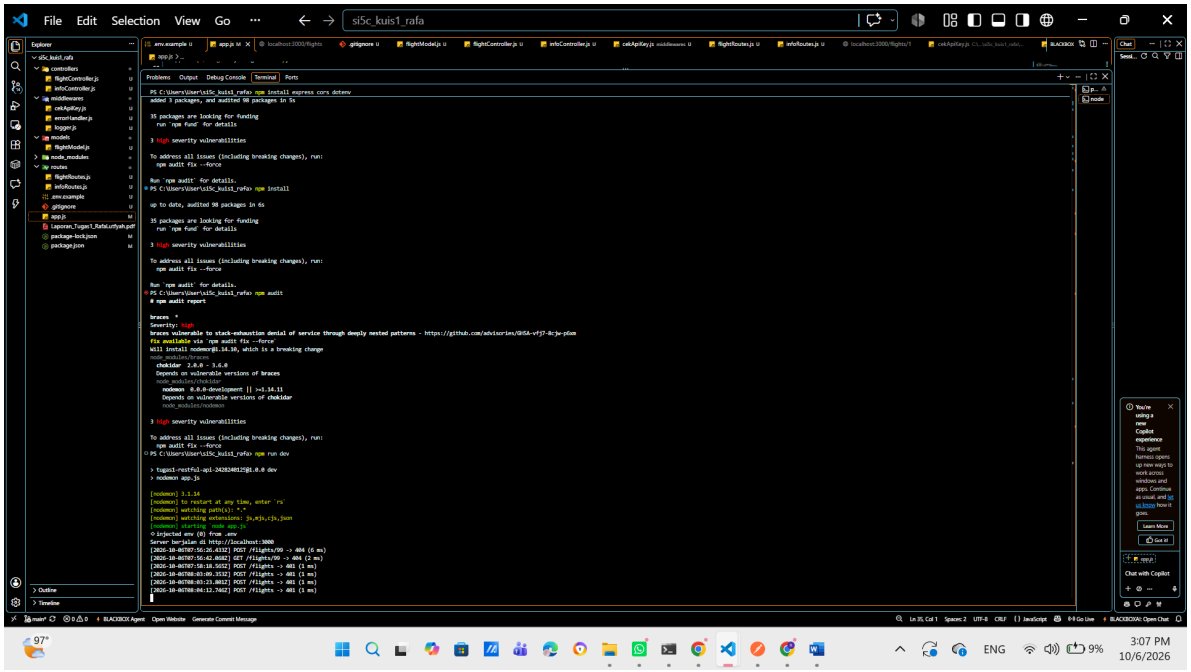
Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
models/flightModel.js	Menyimpan array flights (3 data awal) dan penghitung nextId, serta fungsi getAll, getByAsal, getById, create, update, dan remove. Berkas ini tidak memakai req dan res, jadi hanya mengurus data.
controllers/flightController.js	Berisi getAllFlights, getFlightById, createFlight, updateFlight, dan deleteFlight. Fungsi-fungsi ini membaca req.params, req.query, dan req.body, memanggil validasi() (kodePenerbangan, maskapai, asal, tujuan, jadwalBerangkat wajib diisi), memanggil fungsi Model, lalu mengirim res berstatus 200, 201, 400, atau 404. infoController.js melayani GET /.
routes/flightRoutes.js	Memakai express.Router() untuk memetakan GET / dan GET /:id (terbuka) serta POST /, PUT /:id, dan DELETE /:id (dipasangi cekApiKey) ke fungsi Controller. infoRoutes.js memetakan GET / ke getInfo.
middlewares/logger.js	Mencatat tiap request (waktu, method, URL, status, dan durasi dalam ms) setelah respons selesai lewat event finish, lalu memanggil next().
middlewares/cekApiKey.js	Membaca header x-api-key dan membandingkannya dengan process.env.API_KEY dari .env. Bila kosong atau berbeda, membalas 401 dan tidak memanggil next(), sehingga Controller tidak dijalankan.
middlewares/errorHandler.js	Berisi notFoundHandler (404 'Endpoint tidak ditemukan' untuk rute tak terdaftar) dan errorHandler empat parameter yang mengubah JSON rusak menjadi 400 'Format JSON tidak valid' dan error lain menjadi 500, semuanya berformat { status, message, data }.
app.js	Memuat dotenv, membuat app Express, memasang cors, express.json(), dan logger, lalu me-mount '/' dan '/flights', dan paling akhir notFoundHandler serta errorHandler. app.listen hanya jalan bila NODE_ENV bukan production, dan app diekspor untuk Vercel.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



Gambar 3. Server berhasil dijalankan

Penjelasan:

Nomor port diambil dari process.env.PORT, yang dibaca dotenv dari berkas .env (nilai 3000); jika tidak ada, app.js memakai nilai cadangan 3000. Bila port sudah dipakai aplikasi lain, app.listen gagal dengan error EADDRINUSE dan server tidak menyala, sehingga saya harus mengganti PORT di .env atau menghentikan aplikasi yang memakai port itu.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

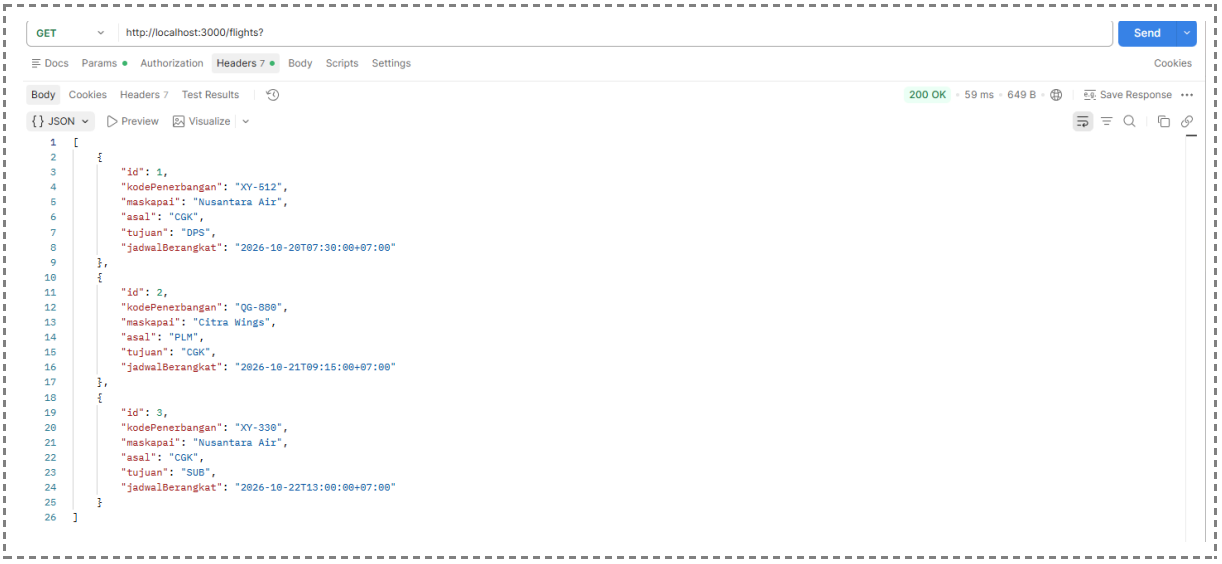
Catatan
Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /flights)

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (*request*)	GET /flights
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	200 OK; larik JSON berisi 3 data awal (id 1, 2, 3).
Lulus?	[x] Ya [ ] Tidak



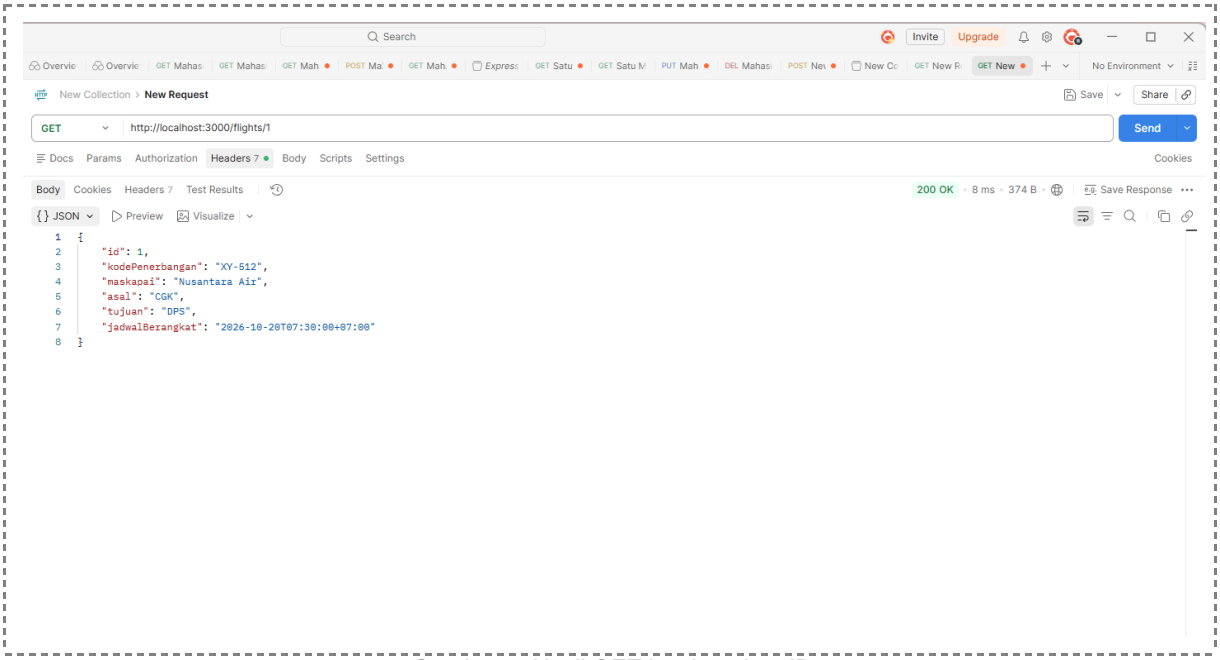
Gambar 4. Hasil GET semua data

Penjelasan:
Status 200 OK karena data berhasil dibaca, dan endpoint ini juga bisa memfilter lewat ?asal=CGK. Alurnya: GET /flights melewati logger, masuk ke flightRoutes (router.get('/')), memanggil getAllFlights di flightController, yang membaca req.query.asal lalu memanggil Flight.getAll() di flightModel (atau Flight.getByAsal(asal) bila ada filter), kemudian res.json mengirim larik datanya.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (*request*)	GET /flights/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	200 OK; objek JSON id 1 (XY-512, CGK ke DPS).
Lulus?	[x] Ya [ ] Tidak



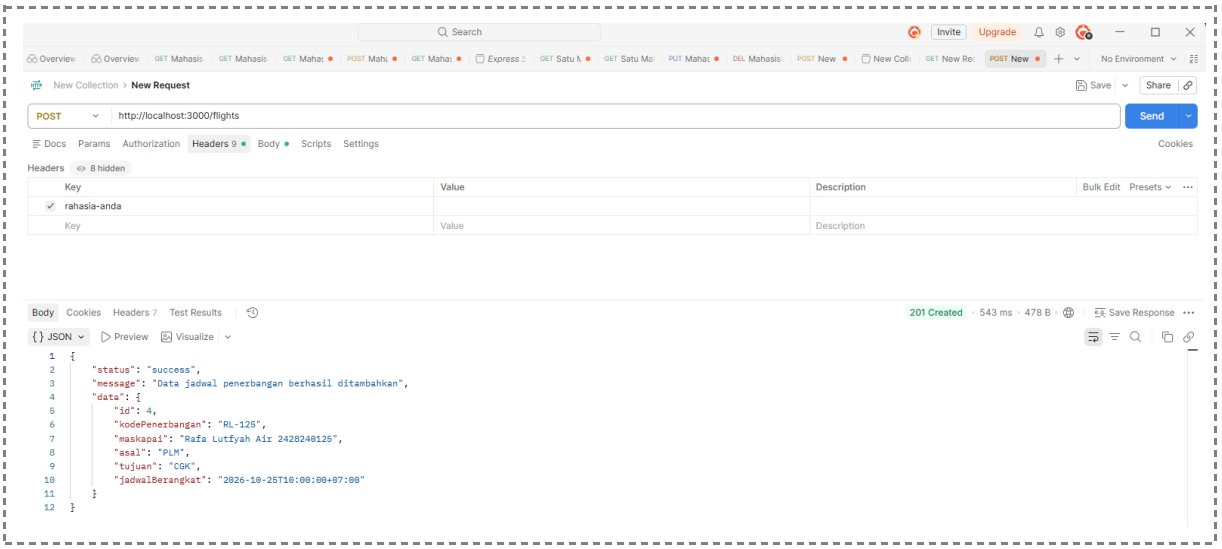
Gambar 5. Hasil GET berdasarkan ID

Penjelasan:
ID pada alamat /flights/1 dibaca lewat req.params.id di getFlightById. Nilai dari alamat selalu berupa string, jadi saya ubah dengan parseInt agar sama dengan tipe angka pada id di array flights; tanpa itu perbandingan f.id === id di Flight.getById tidak akan cocok.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (*request*)	POST /flights
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "kodePenerbangan": "RL-125", "maskapai": "Rafa Lutfyah Air 2428240125", "asal": "PLM", "tujuan": "CGK", "jadwalBerangkat": "2026-10-25T10:00:00+07:00" }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created; message "Data jadwal penerbangan berhasil ditambahkan"; data baru dengan id 4.
Lulus?	[x] Ya [ ] Tidak



Gambar 6. Hasil POST data baru

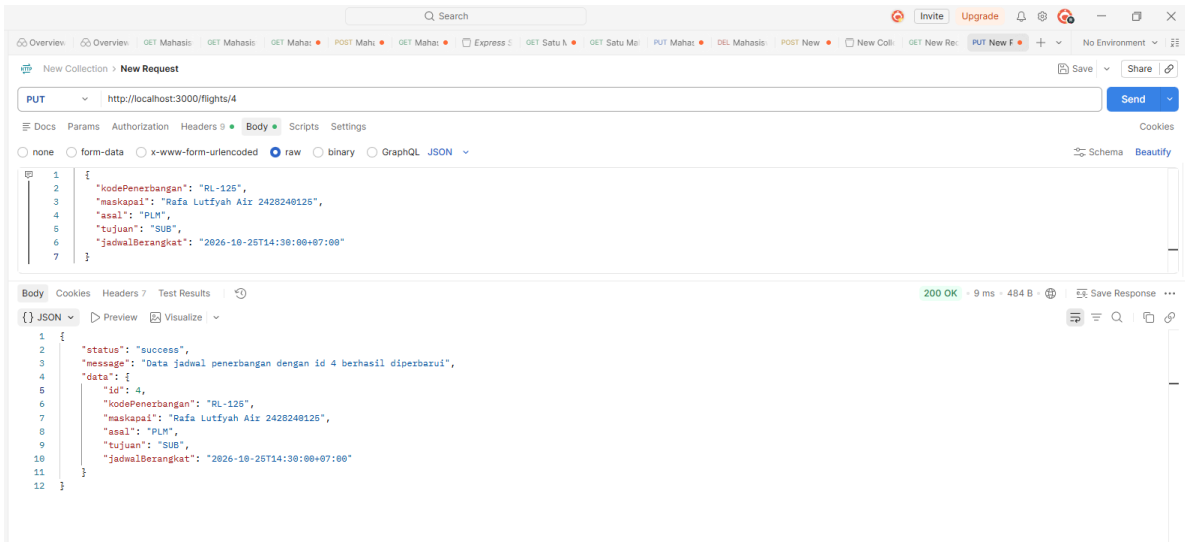
Penjelasan:
Header x-api-key diperiksa middleware cekApiKey yang dipasang pada router.post sebelum Controller, dengan membandingkannya ke API_KEY di .env. express.json() mengubah body JSON menjadi objek di req.body sehingga validasi() dan createFlight bisa membacanya. Status 201 Created dipakai karena request ini membuat resource baru (jadwal id 4), bukan sekadar membaca data seperti 200.

T04 Mengubah data (PUT)

Harus ID data baru (4) pada alamat, header lengkap, body berisi seluruh field (PUT adalah penggantian penuh), dan objek hasil perubahan.

Permintaan (*request*)	PUT /flights/4

Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "kodePenerbangan": "RL-125", "maskapai": "Rafa Lutfyah Air 2428240125", "asal": "PLM", "tujuan": "SUB", "jadwalBerangkat": "2026-10-25T14:30:00+07:00" }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK; message "Data jadwal penerbangan dengan id 4 berhasil diperbarui"; tujuan berubah menjadi SUB dan jadwalBerangkat menjadi 14:30.
Lulus?	[x] Ya [ ] Tidak



Gambar 7. Hasil PUT mengubah data

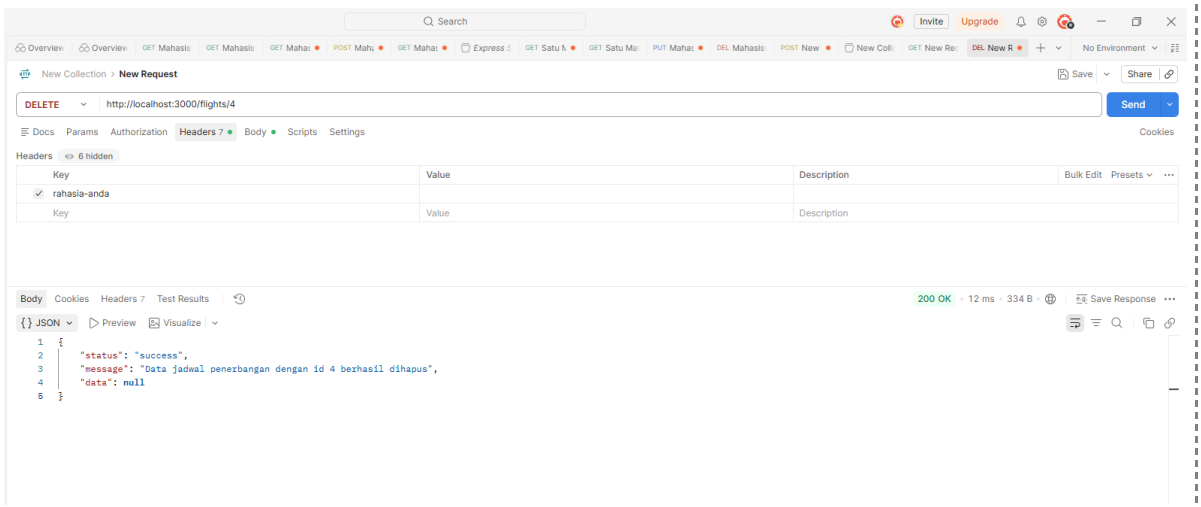
Penjelasan:
Pada PUT /flights/4 saya mengirim seluruh field karena PUT di API ini adalah penggantian penuh. Yang berubah adalah tujuan (CGK menjadi SUB) dan jadwalBerangkat, sedangkan id, kodePenerbangan, maskapai, dan asal tetap sama. Perubahan data dilakukan Model lewat Flight.update, sedangkan Controller (updateFlight) hanya memeriksa id, memvalidasi body, lalu mengirim respons.

T05 Menghapus data (DELETE)

Harus metode DELETE, header x-api-key, status 200, dan respons JSON berisi pesan sukses dengan data: null.

Permintaan (*request*)	DELETE /flights/4
Header	x-api-key: (nilai API_KEY pada .env Anda)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; JSON berisi pesan sukses dan data: null (perilaku DELETE sama seperti Tugas 1).

Status hasil uji	200 OK; message "Data jadwal penerbangan dengan id 4 berhasil dihapus"; data: null.
Lulus?	[x] Ya [ ] Tidak



Gambar 8. Hasil DELETE data

Penjelasan:
Status 200 OK berarti penghapusan berhasil, dan respons berisi pesan 'berhasil dihapus' dengan data: null. Pada Tugas 1 DELETE saya memang membalas 200 beserta pesan, dan karena perilaku API harus tetap sama setelah refactor, saya tidak mengubahnya menjadi 204 No Content (yang berarti sukses tanpa isi respons sama sekali). Penghapusan dilakukan Flight.remove di Model, yang memakai findIndex dan splice.

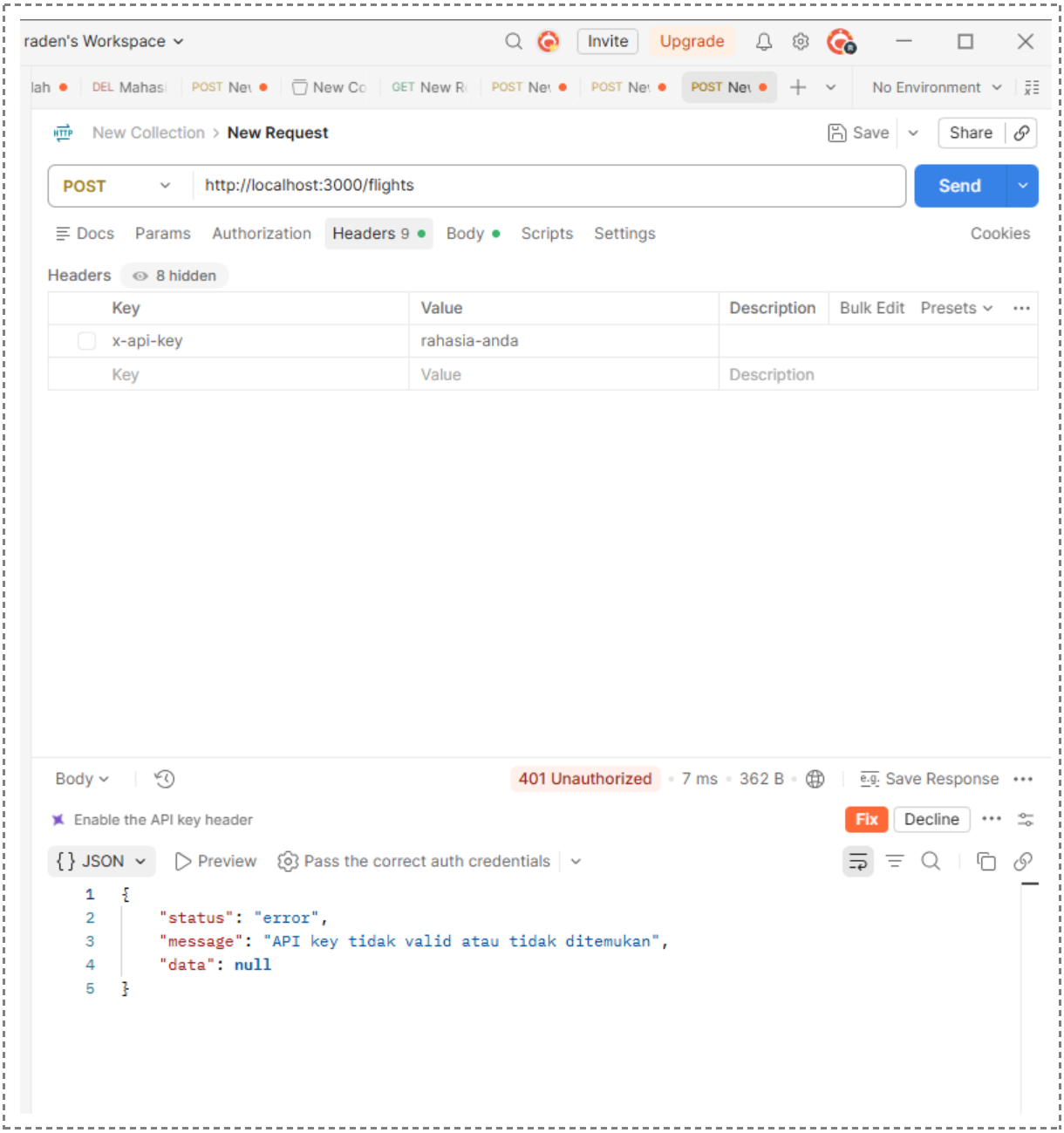
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (*request*)	POST /flights (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ "kodePenerbangan": "RL-125", "maskapai": "Rafa Lutfyah Air 2428240125", "asal": "PLM", "tujuan": "CGK", "jadwalBerangkat": "2026-10-25T10:00:00+07:00" }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized; message "API key tidak valid atau tidak ditemukan"; data: null.
Lulus?	[x] Ya [ ] Tidak



Gambar 9. POST tanpa API key ditolak

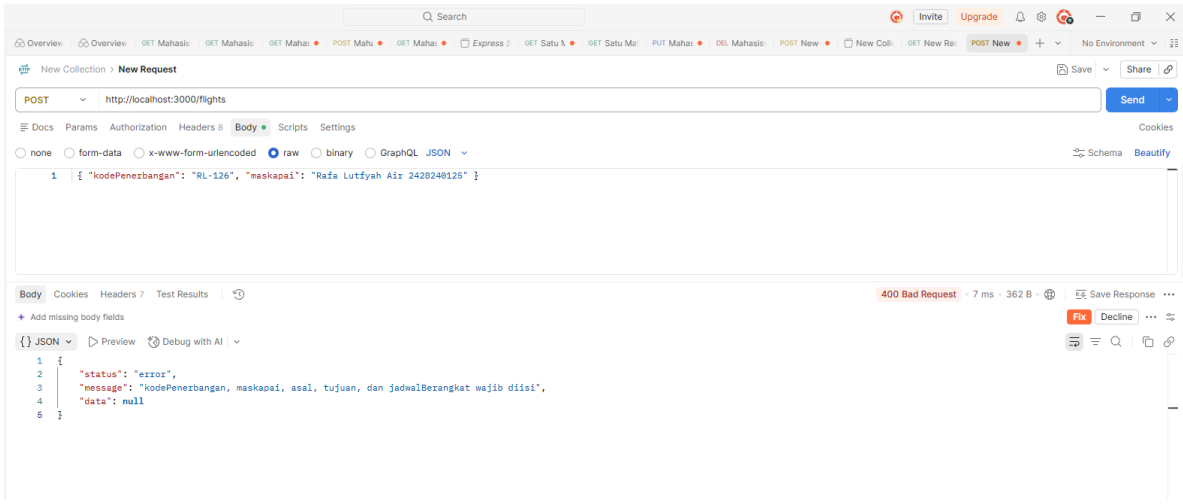
Penjelasan:
Penolakan dilakukan middleware cekApiKey karena header x-api-key tidak dikirim, sehingga ia membalas 401 Unauthorized. cekApiKey tidak memanggil next(), jadi rantai berhenti di situ dan createFlight tidak sempat dijalankan; itu sebabnya data tidak bertambah.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (*request*)	POST /flights
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json

Body	{ "kodePenerbangan": "RL-126", "maskapai": "Rafa Lutfyah Air 2428240125" }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib diisi.
Status hasil uji	400 Bad Request; message "kodePenerbangan, maskapai, asal, tujuan, dan jadwalBerangkat wajib diisi"; data: null.
Lulus?	[x] Ya [ ] Tidak



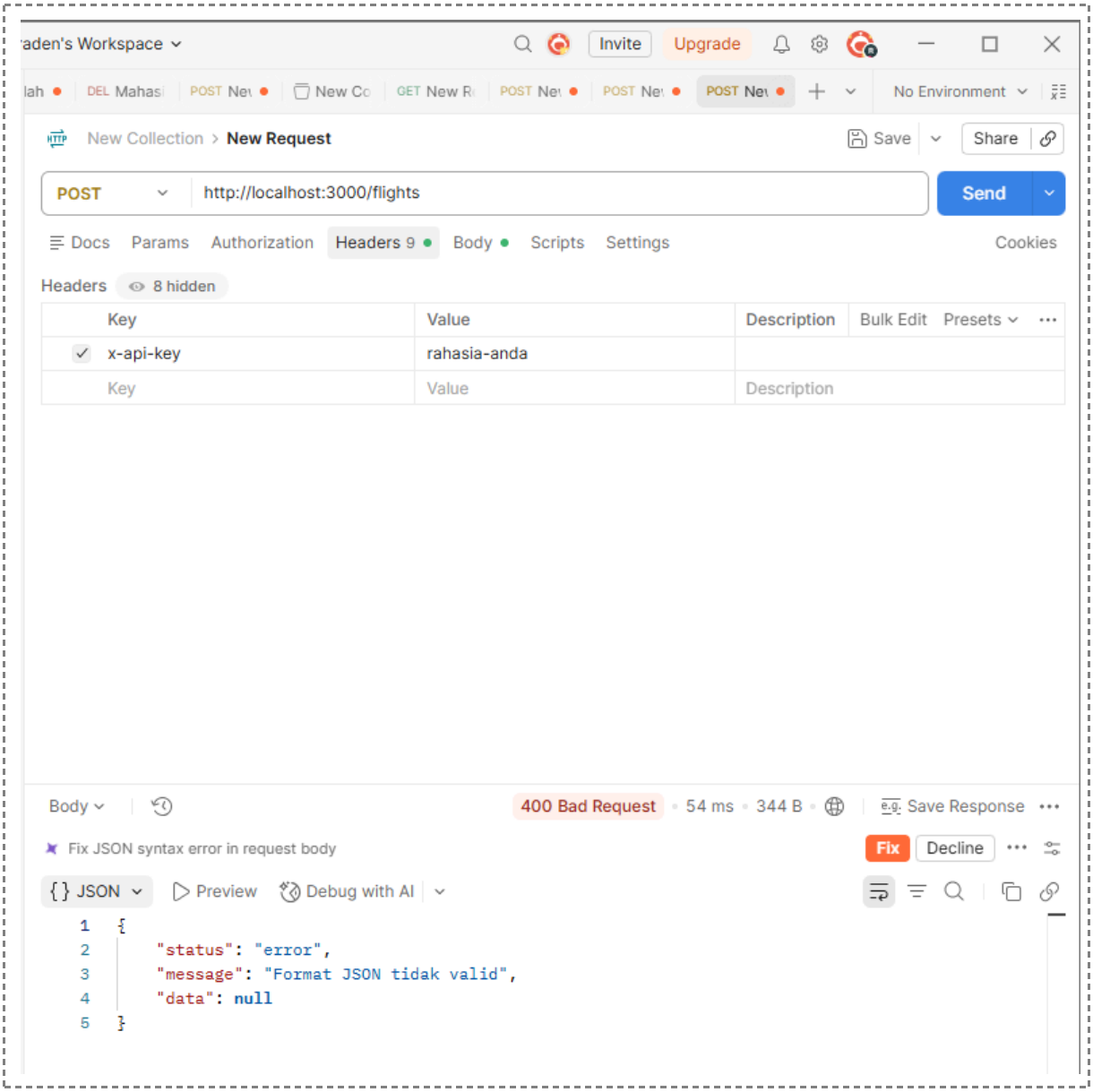
Gambar 10. POST dengan body tidak lengkap

Penjelasan:
Validasi ditulis di Controller, yaitu fungsi validasi() di flightController.js, bukan di Model atau Route. Statusnya 400 Bad Request karena kesalahan ada pada data kiriman client (field wajib tujuan, asal, dan jadwalBerangkat tidak dikirim), bukan masalah di server.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (*request*)	POST /flights
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "kodePenerbangan": "RL-127", "maskapai": "Rafa Lutfyah Air",, "asal": "PLM" }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request; message "Format JSON tidak valid"; data: null.
Lulus?	[x] Ya [ ] Tidak



Gambar 11. POST dengan JSON rusak

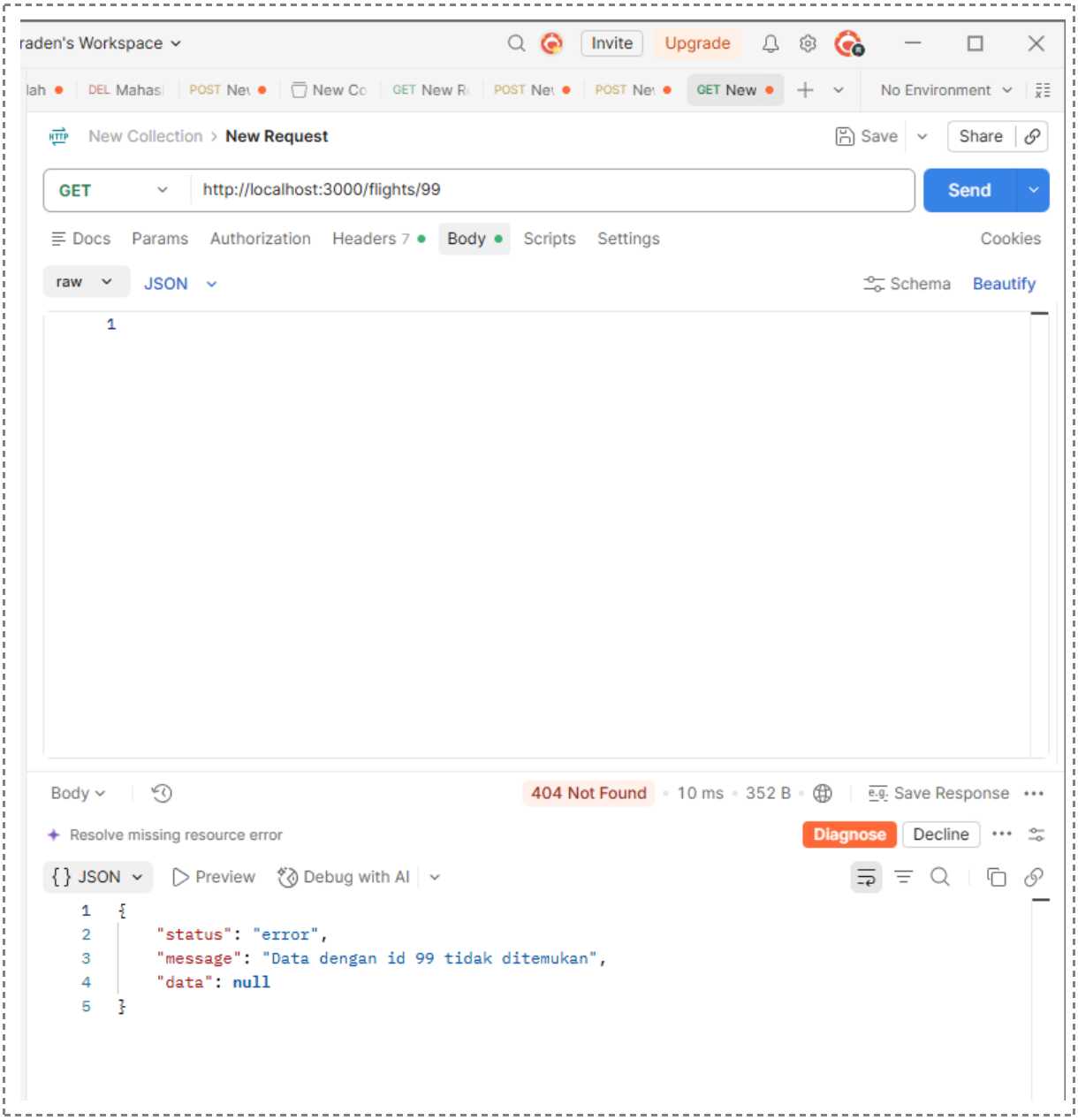
Penjelasan:
JSON rusak dideteksi oleh express.json() saat mem-parsing body, bahkan sebelum request sampai ke route, lalu ia melempar error bertipe entity.parse.failed. Error itu diteruskan ke errorHandler (middleware empat parameter) yang mengenalinya dan membalas 400 dengan { status: 'error', message: 'Format JSON tidak valid', data: null }, sehingga client menerima JSON yang rapi, bukan halaman HTML error bawaan Express.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (*request*)	GET /flights/99
Header	(tidak ada)
Body	(tidak ada)

Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found; message "Data dengan id 99 tidak ditemukan"; data: null.
Lulus?	[x] Ya [ ] Tidak



Gambar 12. GET ID tidak ada

Penjelasan:
Request GET /flights/99 masuk ke getFlightById, id diubah dengan parseInt, lalu Flight.getById(99) mengembalikan undefined. Karena data tidak ada, Controller memanggil helper notFound(res, id) yang mengirim 404 dengan pesan 'Data dengan id 99 tidak ditemukan' dan data: null. Jadi yang mengirim 404 untuk id ini adalah Controller; sedangkan 404 untuk rute yang tidak terdaftar dikirim notFoundHandler di errorHandler.js.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200 OK	Ya
T02	Menampilkan satu data berdasarkan ID	200	200 OK	Ya
T03	Menambah data baru	201	201 Created	Ya
T04	Mengubah data	200	200 OK	Ya
T05	Menghapus data	200	200 OK	Ya
N01	Menambah data tanpa API key	401	401 Unauthorized	Ya
N02	Menambah data dengan body tidak lengkap	400	400 Bad Request	Ya
N03	Mengirim JSON yang rusak	400	400 Bad Request	Ya
N04	Mengambil ID yang tidak ada	404	404 Not Found	Ya

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/raden2428240125/si5c_kuis1_rafa
Pesan commit terakhir	(isi dengan pesan commit terakhir di GitHub Anda)
Folder routes, controllers, models, middlewares ada	[x] Ya [ ] Tidak
.env tidak terunggah (ada di .gitignore)	[x] Ya [ ] Tidak
.env.example ada	[x] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:
Route hanya menghubungkan alamat dan method ke fungsi, contohnya router.post('/', cekApiKey, controller.createFlight) di flightRoutes.js. Controller menangani request dan response, contohnya const pesan = validasi(req.body) lalu res.status(201).json(...) di createFlight. Model hanya mengurus data tanpa req dan res, contohnya flights.push(baru) di fungsi create pada flightModel.js.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:
Dengan error handler terpusat, format error { status, message, data } dan cara menanganinya cukup ditulis di satu tempat (middlewares/errorHandler.js), jadi tidak diulang di tiap route dan mudah diubah. Buktinya pada pengujian N03: JSON rusak, yang tidak saya tangani di route mana pun, tetap dibalas 400 berformat JSON yang rapi, dan rute yang tidak terdaftar juga dibalas 404 'Endpoint tidak ditemukan' oleh notFoundHandler.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan topik Tugas 1 saya	
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	
4	Identitas saya (nama atau NIM) muncul pada data uji POST	
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	
6	Tabel rekapitulasi sesuai dengan tangkapan layar	
7	Tautan repositori pada sampul dapat dibuka dan `.env` tidak terunggah	
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; `.env` tidak terunggah	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		**100**